

SABORES DE CASA

Servicios de Catering y Eventos

DISEÑO TÉCNICO Y PROPUESTA TECNOLÓGICA

Proyecto:

Reto Herramientas IA

Estefanía Llano Pérez

NTT DATA Soluciones IA

4 de mayo de 2026

Versión	Autor	Fecha
1.0	Estefanía Llano Pérez	04/05/2026

Índice del Documento

Índice

1. Introducción	3
1.1. Documentos de referencia	3
1.2. Visión general de la solución	3
1.3. Objetivos técnicos de la solución	3
2. Principios de Diseño y Arquitectura	4
2.1. Principios Fundamentales	4
2.2. Diagrama Conceptual de Capas	4
3. Propuesta Tecnológica (Stack Seleccionado)	4
3.1. Tecnologías de Frontend: Angular y TypeScript	4
3.2. Tecnologías de Backend: Node.js con Express.js	5
3.3. Base de Datos: PostgreSQL	5
4. Diseño en Detalle del Frontend	5
4.1. Estructura General de Módulos (Angular)	5
4.2. Diseño de Componentes Principales	5
4.3. Seguridad Visual y Guards de Navegación	6
5. Diseño en Detalle del Backend	6
5.1. Estructura del Proyecto Node.js (Clean Architecture)	6
5.2. Gestión de Transacciones en Casos Críticos	6
6. Diseño Lógico de la Base de Datos	6
6.1. Diccionario de Tablas Críticas	7
6.2. Consideraciones de Integridad y Rendimiento (Índices)	7
7. Diseño de APIs y Arquitectura de Endpoints	7
8. Seguridad Técnica y Defensa Perimetral	8
8.1. Autenticación (JWT Stateless)	8
8.2. Mitigación de Vulnerabilidades Comunes (OWASP Top 10)	8
9. Rendimiento y Estrategia de Caching	9
10. Estrategia Integral de Pruebas (Quality Assurance)	9
10.1. Pruebas Unitarias (Backend y Frontend)	9
10.2. Pruebas de Integración (APIs)	9
10.3. Pruebas E2E (End-to-End)	10

11.Despliegue, Infraestructura y CI/CD	10
11.1.Contenedorización (Docker)	10
11.2.Integración Continua (CI Pipeline)	10
11.3.Gestión de Entornos	10
12.Conclusión	10

1. Introducción

El presente documento de Diseño Técnico y Propuesta Tecnológica define la solución técnica integral propuesta para el proyecto de centralización operativa de Catering Sabores de Casa. Este documento actúa como puente traduciendo los requisitos de negocio y funcionales, previamente definidos en el Análisis Funcional, en una arquitectura técnica concreta, estableciendo las bases inmutables para el desarrollo, despliegue, seguridad y mantenimiento del sistema.

1.1 Documentos de referencia

Este documento técnico se ha elaborado tomando como base los siguientes entregables, que deben considerarse de lectura obligatoria para una correcta comprensión del alcance global de la solución:

- **Análisis Funcional y de Requisitos (v1.0):** Define el contexto del negocio, la problemática logística actual de la empresa y detalla las necesidades operativas de los distintos perfiles de usuario.
- **Especificación de Interacciones y Flujos (v1.0):** Documento centrado en el comportamiento de usuario y las historias principales (User Stories) que dictan la interacción entre Frontend y Backend.

1.2 Visión general de la solución

La solución propuesta tiene como objetivo principal proporcionar una plataforma web unificada (accesible vía navegador y optimizada para tablets) que permita gestionar de forma estructurada toda la información del negocio: desde la firma inicial del evento por parte de un comercial, pasando por la consolidación de ingredientes para cocina, hasta la generación de cuadrantes de personal por parte de RRHH.

El diseño técnico ha asimilado el contexto real y las restricciones de la organización, caracterizado por:

- Un volumen medio de usuarios concurrentes operando bases de datos críticas.
- Presupuesto ajustado que inhabilita el pago de costosas licencias empresariales.
- Necesidad de no interrumpir la actividad diaria durante el pase a producción.
- Usuarios finales (personal de cocina y sala) con una curva de adopción tecnológica baja.

1.3 Objetivos técnicos de la solución

La solución técnica propuesta persigue los siguientes objetivos inamovibles:

- Centralizar el flujo de información, proporcionando un repositorio único para eventos, clientes, catálogo y personal eventual.
- Garantizar la coherencia e integridad relacional de los datos, evitando bloqueos y pérdidas de información en operaciones concurrentes.
- Aplicar un control de acceso estricto basado en roles (RBAC) desde el núcleo del servidor.

- Soportar operaciones CRUD simultáneas garantizando tiempos de respuesta por debajo de los 2 segundos.
- Utilizar tecnologías de Código Abierto (Open Source) estándar, reduciendo riesgos técnicos y dependencias de proveedores (*vendor lock-in*).

2. Principios de Diseño y Arquitectura

2.1 Principios Fundamentales

El diseño de la solución se basa en paradigmas de ingeniería de software orientados a la longevidad del código:

- **Separación de Responsabilidades (SoC):** La arquitectura separa radicalmente la capa de presentación visual (Frontend), la capa de validación y lógica de negocio (Backend), y la persistencia de datos (BBDD).
- **Simplicidad y Convención sobre Configuración:** El diseño prioriza frameworks que dictan estructuras claras, alineadas con las capacidades de un equipo de desarrollo estándar, evitando complejidades algorítmicas innecesarias.
- **Evolución Incremental y Modularidad:** El código fuente (especialmente en Frontend) estará agrupado por dominios funcionales (Módulo de Cocina, Módulo Comercial), lo que permite inyectar futuras mejoras sin riesgo de colapso del sistema central.

2.2 Diagrama Conceptual de Capas

El flujo general y la topología de la información se orchestra de la siguiente manera:

1. El usuario interactúa con la interfaz gráfica renderizada en su navegador o tablet.
2. El Frontend captura el evento, valida los datos localmente y envía una petición asíncrona HTTP/REST firmada con un Token al Backend.
3. El Backend (API REST) intercepta la petición, verifica la autenticidad del Token y los permisos del rol, y ejecuta la lógica de negocio.
4. El Backend se comunica de forma transaccional con la Base de Datos para persistir o extraer información.
5. El Backend retorna un JSON estructurado y el Frontend actualiza el estado visual de la aplicación.

3. Propuesta Tecnológica (Stack Seleccionado)

Tras evaluar las restricciones económicas y funcionales, se propone un *stack* tecnológico unificado, coherente y robusto, apoyado en el ecosistema TypeScript/JavaScript.

3.1 Tecnologías de Frontend: Angular y TypeScript

Angular ha sido seleccionado como el framework rector para la interfaz de usuario. Al contrario que otras librerías más permisivas, Angular impone una arquitectura *Enterprise* fuertemente orientada a servicios, inyección de dependencias y componentes. Permite organizar la aplicación en módulos encapsulados, lo cual es crítico para aislar, por ejemplo, la lógica de los comerciales de la de RRHH.

El uso inherente de **TypeScript** es innegociable. Aportará tipado estático al código, previniendo errores graves en tiempo de desarrollo (como intentar acceder a propiedades indefinidas de un evento) y facilitando el mantenimiento a medio plazo.

Para la construcción de la interfaz (UI), se emplearán **HTML5**, **SCSS** (para un código CSS estructurado y variable) y el sistema de *grid* de **Bootstrap 5**, garantizando que la aplicación sea responsiva y perfectamente operable en las tablets que utilizará el personal de sala y cocina.

3.2 Tecnologías de Backend: Node.js con Express.js

Node.js actuará como entorno de ejecución en el servidor. Su naturaleza asíncrona orientada a eventos lo hace excepcionalmente eficiente para gestionar un volumen elevado de peticiones concurrentes de entrada/salida, como consultas a la base de datos de los comerciales.

Acoplado a este entorno se desplegará **Express.js**, un framework minimalista de enrutamiento web. Express facilitará la estructuración rápida de los endpoints RESTful, la gestión semántica de errores HTTP y la integración transparente de *middlewares* para validación de esquemas (usando librerías como Joi o Zod) y control de acceso.

3.3 Base de Datos: PostgreSQL

Se rechazan de plano las bases de datos NoSQL para este proyecto. **PostgreSQL** es la elección definitiva como Sistema de Gestión de Base de Datos Relacional (RDBMS). Su robustez y cumplimiento ACID son imprescindibles: un sistema de gestión logística no puede permitirse inconsistencias de datos, documentos huérfanos o fallos en transacciones simultáneas cuando se están asignando menús a un evento de 500 comensales.

4. Diseño en Detalle del Frontend

4.1 Estructura General de Módulos (Angular)

La aplicación Angular se diseña buscando el máximo rendimiento en la carga mediante *Lazy Loading* (carga diferida). La estructura raíz es la siguiente:

- **Core Module:** Contiene servicios *singleton* instanciados una sola vez (como el AuthService, TokenInterceptor para inyectar JWT, y gestores de errores globales).
- **Shared Module:** Agrupa componentes reutilizables en toda la app (botones personalizados, tablas de datos genéricas, modales de confirmación, *pipes* de formato de fechas y divisas).
- **Auth Module:** Pantallas y lógica de inicio de sesión y reseteo de contraseñas.
- **Feature Modules:** Desacoplan la aplicación en grandes bloques de negocio: ComercialModule, ProduccionModule, TalentoModule y CatalogoModule.

4.2 Diseño de Componentes Principales

Los componentes se desarrollarán bajo la filosofía “Smart/Dumb” (Contenedor/Presentacional):

- **Gestión de Eventos:** Un *Smart Component* se encarga de llamar a la API para traer el listado de eventos. Delega la renderización a componentes hijos (tarjetas o tablas). Existirá un componente complejo para el “Formulario de Presupuesto” que gestionará el estado reactivo (*ReactiveForms*) del menú, comensales y fechas con validaciones asíncronas.

- **Consolidado de Cocina:** Componente altamente visual que se actualiza automáticamente. Se nutrirá de *Observables* (RxJS) para procesar los flujos de datos asíncronos y pintar las alertas en tiempo real sin recargar la página.
- **Cuadrantes RRHH:** Interfaz de arrastrar y soltar (Drag & Drop) en el frontend para vincular un trabajador a un turno, lo que desencadenará una petición POST silenciosa al backend.

4.3 Seguridad Visual y Guards de Navegación

El frontend implementará CanActivate Guards. Si un usuario inicia sesión, su token decodificará su rol. Si el usuario (con rol COCINA) altera la URL para ir a /comercial/facturacion, el Guard detectará la infracción, bloqueará la carga del componente, y redirigirá al usuario a una página de 403 No Autorizado. Adicionalmente, se crearán directivas estructurales (ej. *appHasRole="'ADMIN'") para destruir del DOM (y no solo ocultar con CSS) aquellos botones de borrado o edición a los que el usuario no tiene permisos.

5. Diseño en Detalle del Backend

5.1 Estructura del Proyecto Node.js (Clean Architecture)

Para evitar el conocido problema del “código espagueti” en Node.js, el repositorio del backend se estructurará rigurosamente:

- /src/config: Variables de entorno, conexión a BD.
- /src/middlewares: Verificadores de Token (verifyToken), capturadores de errores y validadores de *schemas* de entrada.
- /src/routes: Definición semántica de URIs (/api/eventos).
- /src/controllers: Gestores del flujo de Request y Response HTTP. Su función es extraer los parámetros y llamar a la capa de servicios.
- /src/services: Corazón del backend. Aquí residen los cálculos, como la suma de comensales o la verificación de solapamientos horarios de RRHH.
- /src/models (o repositories): Consultas nativas SQL o abstracciones mediante un ORM ligero (como Prisma o TypeORM) para hablar con PostgreSQL.

5.2 Gestión de Transacciones en Casos Críticos

Se utilizará el control transaccional nativo de PostgreSQL (BEGIN, COMMIT, ROLLBACK). Por ejemplo, al confirmar un evento: 1. Se abre transacción. 2. Se actualiza el estado del evento a “Planificado” en la tabla eventos. 3. Se insertan N filas en la tabla menus_evento. 4. Si el paso 3 falla por un error de conexión, se ejecuta ROLLBACK y el evento vuelve al estado original, garantizando cero inconsistencias en el sistema en vivo.

6. Diseño Lógico de la Base de Datos

La persistencia de datos se modela bajo un esquema relacional estricto, priorizando la Tercera Forma Normal (3NF) para erradicar redundancias.

6.1 Diccionario de Tablas Críticas

Tabla: usuarios

- id_usuario (UUID, Primary Key)
- email (VARCHAR, Unique, Not Null)
- password_hash (VARCHAR, Not Null)
- rol (ENUM: 'ADMIN', 'COMERCIAL', 'COCINA', 'RRHH', Not Null)
- is_active (BOOLEAN, Default True)

Tabla: clientes

- id_cliente (UUID, Primary Key)
- razon_social (VARCHAR, Not Null)
- identificador_fiscal (VARCHAR, Unique, Not Null)
- telefono_contacto (VARCHAR)
- is_active (BOOLEAN, Default True)

Tabla: eventos

- id_evento (UUID, Primary Key)
- id_cliente (UUID, Foreign Key)
- fecha_inicio (TIMESTAMP WITH TIME ZONE, Not Null)
- fecha_fin (TIMESTAMP WITH TIME ZONE, Not Null)
- aforo_estimado (INTEGER, Not Null)
- estado (ENUM: 'BORRADOR', 'PLANIFICADO', 'PREPARACION', 'FINALIZADO', 'CANCELADO')
- notas_alergias (TEXT)

Tabla: menus_evento (**Tabla Puente**)

- id_menu_evento (UUID, Primary Key)
- id_evento (UUID, Foreign Key)
- id_plato_catalogo (UUID, Foreign Key)
- cantidad_raciones (INTEGER, Not Null)

Tabla: cuadrantes_personal (**Tabla Puente**)

- id_cuadrante (UUID, Primary Key)
- id_evento (UUID, Foreign Key)
- id_trabajador (UUID, Foreign Key)
- hora_entrada (TIMESTAMP WITH TIME ZONE)
- hora_salida (TIMESTAMP WITH TIME ZONE)

6.2 Consideraciones de Integridad y Rendimiento (Índices)

Se implementarán **claves foráneas con restricción RESTRICT**, impidiendo borrar físicamente a un cliente si este tiene eventos asociados. Para el borrado se usará exclusivamente la eliminación lógica (`is_active = false`). Para garantizar consultas rápidas (menos de 100ms), se crearán Índices (B-Tree Indexes) sobre columnas altamente consultadas: `fecha_inicio` de los eventos y el `identificador_fiscal` de los clientes.

7. Diseño de APIs y Arquitectura de Endpoints

La comunicación se centralizará a través de endpoints RESTful. A continuación, se detalla una muestra representativa de la API de Eventos:

1. Listar Eventos (Paginado y Filtrado)

- **Método y Ruta:** GET /api/v1/eventos

- **Query Params:** ?estado=PLANIFICADO&fecha_desde=2026-05-01&page=1&limit=20
- **Respuesta Exitosa (200 OK):** JSON con array de objetos y metadatos de paginación.
- **Seguridad:** Requiere token JWT.

2. Alta de Nuevo Evento

- **Método y Ruta:** POST /api/v1/eventos
- **Body Payload (JSON):** { id_cliente: "uuid", fecha_inicio: "...", aforo: 150 }
- **Respuesta Exitosa (201 Created):** Objeto del evento con su nuevo ID generado.
- **Manejo Errores:** 400 Bad Request si la fecha es inválida o falta cliente.

3. Modificar Menús del Evento (Comercial)

- **Método y Ruta:** PUT /api/v1/eventos/{id}/menus
- **Body Payload (JSON):** Array de platos y cantidades actualizadas.
- **Lógica Inyectada:** Si la fecha de ejecución es menor a 48h, el Controlador dispara un evento interno de sistema (EventEmitter) que empuja la notificación por WebSockets al Dashboard de Cocina.

4. Asignación de Trabajador (RRHH)

- **Método y Ruta:** POST /api/v1/cuadrantes
- **Validación de Servicio:** Antes de insertar en BBDD, el servicio ejecuta una consulta que verifica si id_trabajador ya existe en otra fila de cuadrantes_personal cuyas fechas se superpongan con el evento destino. Si es así, retorna 409 Conflict.

8. Seguridad Técnica y Defensa Perimetral

El tratamiento de datos personales y la sensibilidad de las operaciones exigen un blindaje integral a nivel de servidor.

8.1 Autenticación (JWT Stateless)

El sistema descarta el uso de sesiones basadas en servidor (Cookies tradicionales) en favor de *JSON Web Tokens*. El ciclo es el siguiente: 1. El usuario realiza POST /api/auth/login enviando email y password en claro (sobre HTTPS). 2. El servidor coteja contra la base de datos comparando el *hash* generado por la librería Bcrypt (con *salt* autogenerado). 3. Si es correcto, el servidor firma un token JWT que incluye en el *Payload* el ID de usuario, su ROL, y una fecha de expiración (ej. 8 horas). 4. El Frontend guarda el token y lo inyecta en la cabecera Authorization: Bearer <token> en toda petición posterior.

8.2 Mitigación de Vulnerabilidades Comunes (OWASP Top 10)

- **Prevención SQL Injection:** Al usar PostgreSQL con un ORM moderno o consultas parametrizadas, las inyecciones de código malicioso en los inputs de texto quedan automáticamente neutralizadas.
- **Cross-Site Scripting (XSS):** Angular sanitiza de forma nativa todo el contenido renderizado en el DOM, impidiendo la ejecución de scripts inyectados en campos como "Notas del cliente".
- **Cross-Origin Resource Sharing (CORS):** El servidor Node.js se configurará estrictamente mediante el middleware cors() para aceptar tráfico exclusivamente prove-

niente del dominio y puerto en el que esté desplegado el frontend de la compañía (ej. `https://app.saboresdecasa.es`), bloqueando peticiones desde herramientas ajenas.

- **Rate Limiting (Limitador de Tasa):** Para frenar ataques de denegación de servicio (DDoS) o ataques de fuerza bruta contra el Login, se implementará el middleware `express-rate-limit` restringiendo a 20 peticiones por minuto por IP los endpoints críticos.

9. Rendimiento y Estrategia de Caching

Para garantizar un tiempo de respuesta óptimo (< 2 segundos), la arquitectura incorpora medidas de mitigación de latencia:

- **Connection Pooling:** Node.js no abrirá y cerrará conexiones a PostgreSQL con cada petición. Se mantendrá un *pool* (piscina) de conexiones abiertas para reutilizarlas, disminuyendo el *overhead* de la base de datos de manera masiva ante 100+ usuarios concurrentes.
- **Paginación a Nivel de Servidor:** En endpoints pesados como el listado del Histórico de Eventos, la Base de Datos retornará exclusivamente 50 registros por página (`LIMIT 50 OFFSET 0`), evitando colapsar la memoria RAM del servidor y ahorrando ancho de banda en dispositivos móviles.

10. Estrategia Integral de Pruebas (Quality Assurance)

La garantía de un pase a producción libre de fallos críticos reposa en la ejecución de tres niveles de pruebas (Pirámide de Testing):

10.1 Pruebas Unitarias (Backend y Frontend)

- **Backend (Node.js/Jest):** Se escribirán tests aislados para las funciones puras en la capa de Servicios. Ejemplo: Crear un test que alimente al algoritmo de “Cálculo de ratios de camareros” con 345 comensales y espere que la función retorne exactamente el número 23 sin necesidad de conectar a la base de datos real.
- **Frontend (Angular/Jasmine/Karma):** Pruebas sobre la renderización de componentes. Ejemplo: Test que verifique que el botón “Eliminar Evento” no existe en el DOM (DOM Testing) si el servicio inyecta un estado simulado (*mock*) con el rol “COCINA”.

10.2 Pruebas de Integración (APIs)

Se utilizará la librería *Supertest* combinada con *Jest* para levantar un servidor efímero acoplado a una base de datos de pruebas (en memoria o vacía).

- **Caso de éxito:** Se simula un `POST /eventos` completo, verificando el código HTTP 201 y comprobando que el registro existe físicamente en la BD.
- **Caso de error:** Se simula enviar un `POST` sin incluir el token JWT en las cabeceras, exigiendo que el servidor aborte y responda un riguroso 401 `Unauthorized`.

10.3 Pruebas E2E (End-to-End)

Se empleará el framework *Cypress* para simular automatizadamente a un humano moviendo el ratón y tecleando en la aplicación desplegada en un entorno controlado (Staging). Cypress abrirá el navegador, escribirá usuario y clave, hará login, navegará a “Nuevo Evento”, tecleará los datos, guardará y verificará que el evento aparece en el listado general.

11. Despliegue, Infraestructura y CI/CD

El flujo de entrega del código pasará de ser manual a estar totalmente automatizado y estandarizado, eliminando el riesgo de errores en la publicación de actualizaciones (releases).

11.1 Contenedorización (Docker)

Toda la aplicación será empaquetada. Se crearán dos archivos *Dockerfile*: uno para compilar y servir el frontend de Angular (usando un servidor ligero como Nginx), y otro para el entorno Node.js del backend. Mediante *docker-compose*, la infraestructura entera (Frontend + Backend + PostgreSQL) podrá levantarse en el servidor local de cualquier desarrollador con un solo comando, asegurando paridad milimétrica entre la máquina del programador y el servidor en la nube.

11.2 Integración Continua (CI Pipeline)

Se configurará un pipeline en el repositorio de control de versiones (ej. GitHub Actions, GitLab CI). Cada vez que un desarrollador intente integrar nuevo código (*Push* o *Pull Request*), la plataforma ejecutará en la nube: 1. Análisis de código estático (Linting / ESLint). 2. Compilación del proyecto (Build para verificar errores de sintaxis TypeScript). 3. Ejecución de la batería de Pruebas Unitarias y de Integración. Si alguno de estos pasos falla, el código será rechazado automáticamente y no podrá avanzar hacia el servidor de Producción.

11.3 Gestión de Entornos

La infraestructura final contará con tres entornos aislados y diferenciados:

- **Development (DEV):** Entorno inestable, conectado a bases de datos con datos ficticios. Uso exclusivo de programadores.
- **Staging / Pruebas (UAT):** Entorno de Pre-producción. Réplica técnica exacta del entorno real. Se utilizará para que la Dirección y usuarios clave realicen las validaciones de aceptación final.
- **Production (PROD):** El entorno sagrado, protegido, con monitorización continua, escalabilidad horizontal bajo demanda (Auto-Scaling en AWS/Render) y respaldos automatizados de base de datos (Backups) cada 24 horas.

12. Conclusión

El presente diseño técnico proporciona el cimiento inquebrantable que Catering Sabores de Casa requiere para acometer su centralización operativa de forma segura y exitosa.

La estructuración en tres capas separadas, la adopción de frameworks empresariales modernos (Angular y Node.js), y la estricta persistencia relacional en PostgreSQL conforman un núcleo invulnerable ante las altas demandas transaccionales del negocio. Por otra parte, la aplicación de medidas de seguridad profundas, pruebas automatizadas en cada despliegue e infraestructura en contenedores aseguran que la plataforma no solo absorberá la operativa actual sin fisuras, sino que crecerá exponencialmente adaptándose a futuras expansiones del negocio sin sufrir la degradación técnica y la deuda acumulada de su actual sistema ofimático.